ACCELEROMETER & GYROSCOPE ---- data flow

INPUT
Physical force (linear acceleration / impact) + Angular rotation (body orientation)
Accelerometer (linear) - Gyroscope (angular)

Accelerometer: MEMS mass moves linearly → ΔC = C2−C1 → voltage → 16-bit ADC
OUTPUT: *ax_raw / ay_raw / az_raw*

Gyroscope: Microscopic structures vibrate → Coriolis force deflects them → angular velocity → 16-bit ADC
OUTPUT: *gx_raw / gy_raw / gz_raw*

All 6 values transmitted via I2C → ESP32

STEP 2 — CONVERSION (ESP32)

- Raw values = integer data (no physical meaning yet)
- Each value is divided by a sensitivity factor
- Sensitivity depends on the selected measurement range (according to the datasheet of the sensor)
- Convert to real units
- **Output:** Acceleration → g / Angular velocity → °/s (example: ax=ax raw/ sensitivity factor)

STEP 3 — FILTERING (ESP32)

- Raw signal → contains small noise
- Noise → may cause false detections
- Real impact → higher amplitude, still visible
- Filtering → smooths noise, keeps real events

Formula:

$ax_filtered = (ax_current + ax_previous) / 2$, $ay_filtered = (ay_current + ay_previous) / 2$, $az_filtered = (az_current + az_previous) / 2$

(same goes with gx)

Output:
Filtered acceleration → ready for next step

If higher precision is required, more advanced filtering methods can be implemented for better event recognition

STEP 4 — CALCULATION (ESP32)

accelerometer

- Falls can happen in any direction
- One axis alone → risk of missing event
- Magnitude = merge of all 3 axes
- Shows total impact strength

Formula:

$$Magnitude = \sqrt{ax^2 + ay^2 + az^2}$$

Output:
Direction-independent impact intensity

Gyroscope

- Gyroscope gives rotation speed (°/s), not angle
- ESP32 accumulates rotation over time → total angle
- Concept: like distance = speed × time
- Each small time step adds rotation to previous angle
- Result = body's orientation change since start

Formula (concept):

$$Angle = Previous\ Angle + (Rotation\ Speed \times Time\ Step)$$

Output:
Accumulated body angle → used as condition for fall detection

STEP 5 — DECISION (ESP32)

OUTPUT : body_angle in degrees → Combined with accelerometer → FALL DETECTED TRUE/FALSE → Communication Layer → Caregiver notified

HR/ HRV & spo2---- data flow

INPUT:

- Red LED (660 nm) + IR LED (940 nm) Both emit light into the skin

STEP 1 — ACQUISITION

- Sensor alternates LEDs (Red / IR)
- Red LED ON → photodetector reads → **raw_red** saved
- IR LED ON → photodetector reads → **raw_ir** saved
- Photodiode converts **light intensity** → **current(**the flow of electric charge (electrons))
- TIA converts **current** → **voltage**
- 18-bit ADC converts **voltage** → **digital value**

Meaning of raw values:

- **raw_red / raw_ir = amount of light detected after absorption by blood**

STEP 2 — SIGNAL SEPARATION

- ESP32 receives **raw_red** and **raw_ir**
- For each signal:
  - **DC** = average over time → stable level → absorption by fixed tissues
  - **AC** = current value – DC → rhythmic variations → absorption by pulsating blood

Outputs:

- **AC_ir** → used for HR & HRV
- **AC_red + AC_ir** → used together for SpO_2
- **DC_red + DC_ir** → used together for SpO_2

STEP 3 —CALCULATION

HR/HRv

-Uses AC_ir only (IR penetrates deeper → cleaner signal)

-ESP32 detects peaks in AC_ir (heartbeat pulses)-

Each peak → timestamp recorded

Calculations:

- **IBI (Inter-Beat Interval)** = time between two peaks
- **HR (bpm)** = 60000 / IBI
- **HRV** = variation between successive IBIs

spo2

Uses **ra**w_red + raw_ir together

- **hemoglobin absorbs differently at 660 nm (red) vs 940 nm (IR)**
  - **Oxygenated Hb → absorbs less red, more IR**
  - Deoxygenated Hb → absorbs more red, less IR

Calculation:

- Ratio

$$R = (AC_{red} / DC_{red}) \div (AC_{ir} / DC_{ir})$$

- **High R → more deoxygenated blood → low SpO_2**
- **Low R → more oxygenated blood → high SpO_2**

STEP 4 — DECISION (ESP32)

OUTPUT : o2 values → HR/HRV values →physiological alert → Communication Layer → Caregiver notified

glucose estimation ---- data flow

INPUT
IR LED (940 nm) + Red LED (660 nm) → light emitted into skin
Sensor: MAX30102

STEP 1 — ACQUISITION

- MAX30102 alternates Red/IR LEDs
- Light penetrates skin → blood vessels
- Glucose changes blood viscosity → alters absorption
- Photodetector captures returned light
- Internal ADC converts to digital values
- Data sent

STEP 2 — DC / AC SEPARATION

- DC = stable average → fixed tissues
- AC = rhythmic variations → pulsating blood

Output: AC_ir / DC_ir, AC_red / DC_red

STEP 3 — FEATURE EXTRACTION

- **Feature 1 — IR/Red Ratio:**
  $(AC_{ir}/DC_{ir}) \div (AC_{red}/DC_{red})$ → absorption ratio → linked to viscosity → glucose effect
- **Feature 2 — Signal Variability:**
  Standard deviation of AC_ir → pulse variability → affected by glucose
- **Feature 3 — Waveform Slope:**
  Rise speed of AC_ir peak → slope changes with blood thickness

Output: [ratio, variability, slope] → 3-value feature vector

STEP 4 — ML MODEL

Model trained on PPG + true glucose data

- **Output:** glucose_estimated (mg/dL)

skin Temperature -- data flow

INPUT:

- Body heat naturally produced by human metabolism

STEP 1 — ACQUISITION

- The sensor (MAX30205)is placed directly on the skin — heat transfers by direct contact
- Internal resistive element changes electrically with temperature
- Internal 16-bit sigma-delta ADC converts the analog signal → digital value
- Sent via I2C → esp32

Output: temp_raw (digital value)

STEP 2 — conversion

- esp32 receives temp_raw
- Applies datasheet conversion factor
- Temperature (°C) = temp_raw × 0.00390625
- Output: temperature (°C)

STEP 3 — FILTERING

STEP 4 — DECISION (ESP32)

Temperature (°C) → Threshold exceeded → ALERT_PHYSIOLOGICAL = TRUE → Communication Layer → Caregiver notified

GPS --data flow

INPUT:

Satellite GPS signals
Sensor: **GPS module**

STEP 1 — ACQUISITION

- GPS module receives signals from satellites
- Position calculated by triangulation (≥ 3 satellites)
- Data sent as NMEA
- Example:$GPRMC,123519,A,3607.5234,N,00302.5021,E,...

Output: raw NMEA message

STEP 2 — PROCESSING

- ESP32 splits NMEA message by commas
- Standard format → fixed positions:
  - Latitude = field 4 → 3607.5234 N
  - Longitude = field 6 → 00302.5021 E
- Converted to decimal degrees:
  - Latitude = 36.7525
  - Longitude = 3.0420

Output: latitude + longitude (decimal values)

STEP 3 — ATTACHMENT TO ALERTS

- GPS position automatically added to each alert:
  - Fall detected → alert + lat/lon
  - Emergency button → alert + lat/lon
  - Physiological alert → alert + lat/lon

Output: alert + GPS coordinates

STEP 4 —DISPLAY TO CAREGIVER

- Mobile app receives lat/lon via GSM
- Passed to Google Maps API / OpenStreetMap
- Point displayed on map:
  - lat = 36.7525, lon = 3.0420
- Caregiver sees exact location

Output: real-time latitude + longitude → included in alerts → shown on caregiver's map

sleep tracker --data flow

INPUT:

Extracted physiological signals: **HR + HRV + SpO_2 (%)**

STEP 1 — ACQUISITION

- ESP32 receives continuous HR + SpO_2 values
- Moving average filter → remove noise
- Combined features → sleep stage indicators
- Stable HR + stable SpO_2 → **Deep sleep**
  Moderate HRV + stable SpO_2 → **Light sleep**
  Irregular HR + SpO_2 drops → **Wake / disturbance**

STEP 2 — PROCESSING

- sleep_stage timeline → Communication Layer → Caregiver / health app notified

HEART RATE / HRV / SpO2 — Physiological Monitoring

MAX30102 · I2C · 100 Hz · ESP32

INPUT
LED Red (660nm) + LED Infrared (940nm) emitted into the skin
MAX30102 · raw_red + raw_ir · 18-bit ADC · I2C · 100 Hz

I2C bus

STEP 1 — ACQUISITION

MAX30102 alternates the two LEDs — never simultaneously on
LED Red ON → photodetector reads → raw_red saved
LED IR ON → photodetector reads → raw_ir saved

raw_red (660nm): deoxygenated Hb absorbs MUCH red / HbO2 absorbs LITTLE red
raw_ir (940nm): HbO2 absorbs MUCH IR / deoxygenated Hb absorbs LITTLE IR

18-bit ADC → transmitted via I2C → ESP32 at 100 Hz

STEP 2 — DC / AC SEPARATION (ESP32)

DC = average over time window = stable baseline = fixed tissues (skin, bone, fat)
AC = current value − DC = small rhythmic variations = pulsing blood only

DC_ir = average(raw_ir) · AC_ir = raw_ir − DC_ir (every instant)

OUTPUT: AC_ir → HR and HRV · AC_red + AC_ir + DC_red + DC_ir → SpO2

STEP 3 — HR & HRV CALCULATION (ESP32)

Uses AC_ir only — IR penetrates deeper → stronger cleaner signal

Peak detected when: AC(t) > AC(t−10ms) AND AC(t) > AC(t+10ms) AND AC(t) > min threshold

IBI = timestamp($peak_2$) − timestamp($peak_1$) in ms
HR = 60 000 / IBI in bpm (Example: IBI=800ms → HR=75 bpm)

HRV = variation between successive IBI values
Stable IBI → low HRV → calm · Variable IBI → high HRV → stress / cardiac risk

STEP 4 — SpO2 CALCULATION (ESP32)

Uses both raw_red AND raw_ir — different absorption at 660nm vs 940nm:
Hb deoxygenated: MUCH red / LITTLE IR
HbO2 oxygenated: LITTLE red / MUCH IR

R = (AC_red / DC_red) / (AC_ir / DC_ir)
R high → deoxygenated → SpO2 low · R low → oxygenated → SpO2 high

SpO2 (%) = 110 − 25 × R Normal: 95%–100% Alert threshold: < 92%

STEP 5 — DECISION (ESP32)

HR < low threshold → bradycardia → ALERT
HR > high threshold → tachycardia → ALERT
Irregular rhythm → arrhythmia → ALERT
Abnormal HRV → cardiac danger → ALERT

SpO2 < 92% → hypoxia → IMMEDIATE ALERT
SpO2 decreasing → preventive alert before collapse

GSM module

OUTPUT
HR (bpm) + HRV + SpO2 (%) → ALERT_PHYSIOLOGICAL = TRUE → GSM → Mobile App → Caregiver notified

SKIN TEMPERATURE — MLX90614 Contactless IR

MLX90614 · Contactless IR · I2C → ESP32

INPUT
Infrared radiation naturally emitted by the skin — no contact required
MLX90614 · Contactless · Thermopile sensor · I2C

I2C bus

STEP 1 — ACQUISITION

Every object emits IR radiation proportional to its temperature
MLX90614 internal thermopile (series of thermocouples) captures IR → voltage → ADC → I2C → ESP32
OUTPUT: temp_raw → ESP32

STEP 2 — CONVERSION — Kelvin to Celsius (ESP32)

Sensor measures in Kelvin — factor 0.02 from MLX90614 datasheet

temperature (°C) = (temp_raw × 0.02) − 273.15

0.02 = sensor resolution · 273.15 = Kelvin to Celsius offset

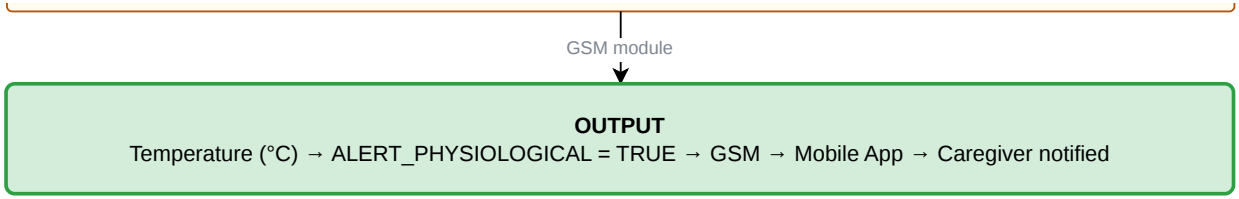
STEP 3 — FILTERING — Sliding average (ESP32)

Sliding average over last 10 measurements eliminates ambient temperature spikes

temp_filtered = $(t_1 + t_2 + ... + t_{10}) / 10$
→ outputs smooth stable temperature value

STEP 4 — DECISION (ESP32)

temp > 38°C → fever → fall risk → ALERT
temp < 35°C → hypothermia → IMMEDIATE ALERT
Sudden drop + abnormal HR + SpO2 → circulatory shock → CRITICAL ALERT


GSM module
OUTPUT
Temperature (°C) → ALERT_PHYSIOLOGICAL = TRUE → GSM → Mobile App → Caregiver notified

GLUCOSE ESTIMATION — Non-invasive PPG + ML

MAX30102 · PPG morphology · Linear Regression · ESP32

INPUT
Same MAX30102 as HR/SpO2 — glucose changes blood viscosity → changes light absorption pattern
MAX30102 · raw_ir + raw_red · 100 Hz · PPG morphology

↓

STEP 1 — ACQUISITION

Same as HR/SpO2: MAX30102 alternates Red and IR LEDs
Key principle: glucose raises blood viscosity → thicker blood changes pulse propagation
OUTPUT: raw_ir + raw_red → ESP32 (100 Hz, I2C)

↓

STEP 2 — DC / AC SEPARATION (ESP32)

DC = stable baseline = fixed tissues · AC = rhythmic variations = pulsing blood
OUTPUT: AC_ir / DC_ir / AC_red / DC_red

↓

STEP 3 — FEATURE EXTRACTION (ESP32)

3 features = input vector to ML model:

Feature 1 — Ratio: ratio = (AC_ir/DC_ir) / (AC_red/DC_red)
→ light absorption changes with viscosity → viscosity changes with glucose

Feature 2 — Variability: std_deviation(AC_ir over time window)
→ high glucose → thick blood → different pulse variations

Feature 3 — Slope: rising speed of AC_ir peak (valley → peak)
→ thick blood → slower rise → different slope

OUTPUT: [ratio, variability, slope]

↓

STEP 4 — ML MODEL — Linear Regression (ESP32)

Trained on computer (Python) on thousands of PPG + finger-prick glucose measurements

Coefficients a, b, c, d saved in model.csv on SD card — loaded by ESP32 at startup

Prediction every 5 seconds:
glucose (mg/dL) = (a × ratio) + (b × variability) + (c × slope) + d

Scientific references:
IEEE: ieeexplore.ieee.org/document/10463200
Nature: nature.com/articles/s41598-024-84265-8
IEEE dataset: ieeexplore.ieee.org/document/9063187

↓

STEP 5 — DECISION (ESP32)

glucose < 70 mg/dL → Hypoglycemia → IMMEDIATE ALERT
glucose > 180 mg/dL → Hyperglycemia → IMMEDIATE ALERT
70 ≤ glucose ≤ 180 → Normal range → continuous monitoring

↓ GSM module

OUTPUT
Glucose (mg/dL) → ALERT_GLUCOSE = TRUE → GSM → Mobile App → Caregiver notified

GPS & LOCATION TRACKING

GPS Module · NMEA format · UART → ESP32 · 1 Hz

INPUT
GPS satellite signals — triangulation from minimum 3 satellites
GPS Module · NMEA standardized text format · UART · 1 Hz

UART

STEP 1 — ACQUISITION

GPS module calculates position by triangulating satellite signals
Sends NMEA standardized text format via UART → ESP32 at 1 Hz

Example: $GPRMC,123519,A,3607.5234,N,00302.5021,E,...

STEP 2 — NMEA PARSING (ESP32)

ESP32 splits NMEA string by commas — standardized same order always
Latitude = angle N/S from equator · Longitude = angle E/W from Greenwich
Two values = unique point on the entire planet

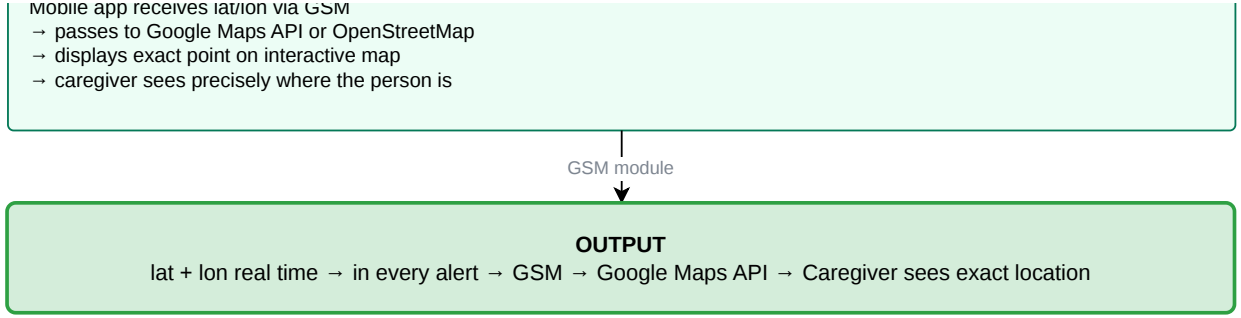
Position 4 → latitude = 3607.5234 → decimal: 36.7525°N
Position 6 → longitude = 00302.5021 → decimal: 3.0420°E

STEP 3 — ATTACHED TO EVERY ALERT

GPS coordinates included automatically in EVERY alert:
Fall detected → alert + lat/lon
Emergency button → alert + lat/lon
Physiological alert → alert + lat/lon
Glucose alert → alert + lat/lon

STEP 4 — DISPLAY FOR CAREGIVER

Mobile app receives lat/lon via GSM


Mobile app receives lat/lon via GSM
→ passes to Google Maps API or OpenStreetMap
→ displays exact point on interactive map
→ caregiver sees precisely where the person is
GSM module
OUTPUT
lat + lon real time → in every alert → GSM → Google Maps API → Caregiver sees exact location

EMERGENCY BUTTON — Instant Alert

Push Button · GPIO Hardware Interrupt → ESP32

INPUT
Physical press on the emergency button
Push Button · GPIO pin · Hardware interrupt · Microseconds reaction

GPIO pin

STEP 1 — ACQUISITION — GPIO interrupt

Button connected to GPIO pin of ESP32
At rest → GPIO = LOW (0V)
Button pressed → GPIO = HIGH (3.3V)

ESP32 detects change via hardware interrupt
→ INSTANTANEOUS reaction (microseconds)

OUTPUT: GPIO = HIGH → emergency trigger fired immediately

STEP 2 — ALERT CONSTRUCTION (ESP32)

Alert package built immediately:
→ type: EMERGENCY_BUTTON_PRESSED
→ position: latitude + longitude (real-time GPS)
→ vitals: HR + SpO2 + Temperature at moment of press
→ timestamp: exact date and time

GSM module

OUTPUT
Complete alert → GSM → Mobile App → Caregiver notified immediately + GPS location on map

MEDICATION REMINDER — Time-Based Trigger

Internal Clock · No Physical Sensor · ESP32

INPUT
ESP32 internal clock — medication schedules defined by caregiver in mobile app
Internal clock · No sensor · Time-based trigger

↓

STEP 1 — TIME CHECK

ESP32 internal clock tracks time continuously
Medication schedules received from mobile app and stored in memory
→ current time read every second

↓

STEP 2 — COMPARISON (ESP32)

current_time == medication_scheduled_time ?

YES → trigger reminder
NO → continue, check next second

↓

STEP 3 — NOTIFICATION CONSTRUCTION

Notification built:
→ type: MEDICATION_REMINDER
→ scheduled: defined time
→ timestamp: current time

↓ GSM module

OUTPUT
Reminder notification → GSM → Mobile App → Caregiver notified

SLEEP TRACKER — Anomaly Detection During Sleep

MAX30102 + MPU-6050 · I2C → ESP32 · 100 Hz

INPUT
raw_ir + raw_red (MAX30102) + ax/ay/az (MPU-6050)
MAX30102 · MPU-6050 · I2C · 100 Hz

I2C bus

STEP 1 — ACQUISITION

MAX30102 measures HR and SpO2 continuously (same process as HR/SpO2 flow)
MPU-6050 measures ax/ay/az continuously for movement detection
OUTPUT: HR (bpm) + SpO2 (%) + ax/ay/az → ESP32 (100 Hz)

STEP 2 — MOVEMENT CALCULATION (ESP32)

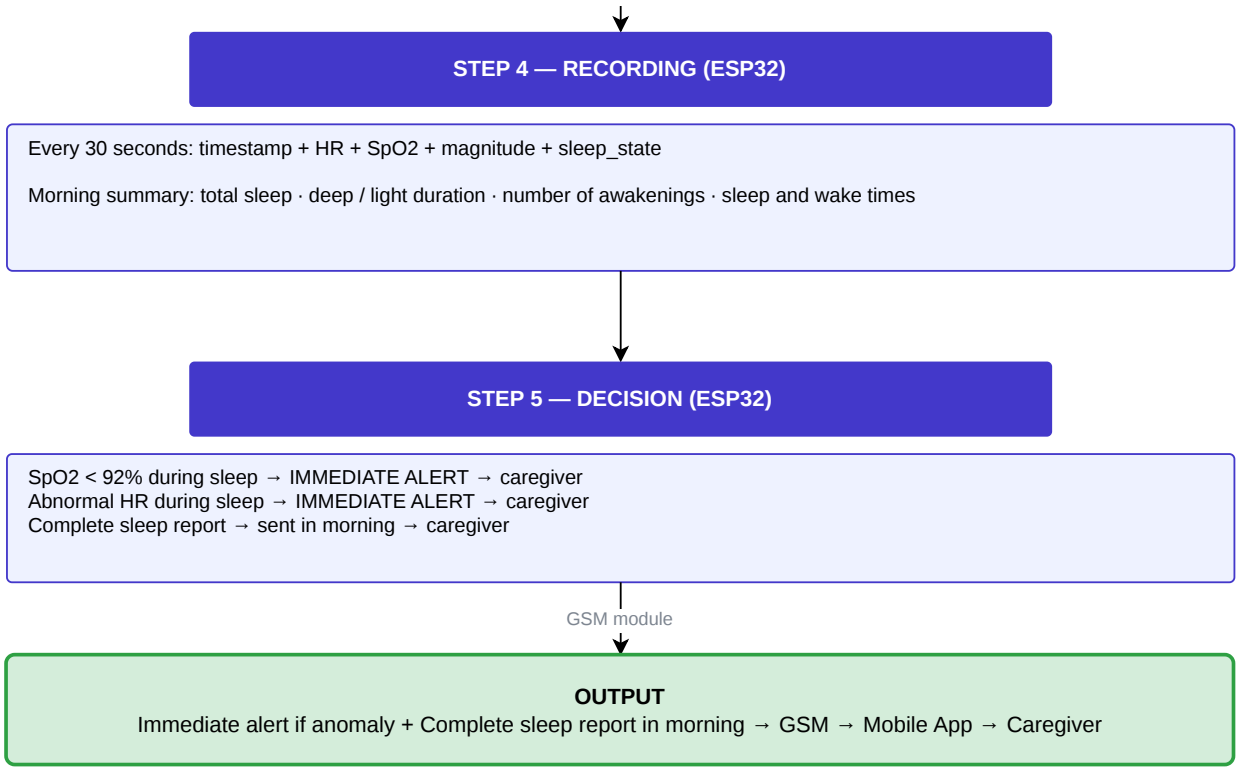
magnitude = sqrt($ax^2 + ay^2 + az^2$)
Average calculated every 30 seconds

Low magnitude → little movement → person sleeping
High magnitude → much movement → person awake / restless

STEP 3 — SLEEP CLASSIFICATION (ESP32)

HR + SpO2 + magnitude combined every 30 seconds:

State	Heart Rate	SpO2	Magnitude	Action
Deep sleep	Stable and low	95–100%	Very low	Normal
Light sleep	Slightly variable	Normal	Low	Normal
Night awakening	Increases	Normal	High	Logged
Sleep apnea	Any	**< 92%**	Any	**ALERT NOW**
Cardiac anomaly	**Very irregular**	Any	Any	**ALERT NOW**


STEP 4 — RECORDING (ESP32)
Every 30 seconds: timestamp + HR + SpO2 + magnitude + sleep_state
Morning summary: total sleep · deep / light duration · number of awakenings · sleep and wake times
STEP 5 — DECISION (ESP32)
SpO2 < 92% during sleep → IMMEDIATE ALERT → caregiver
Abnormal HR during sleep → IMMEDIATE ALERT → caregiver
Complete sleep report → sent in morning → caregiver
GSM module
OUTPUT
Immediate alert if anomaly + Complete sleep report in morning → GSM → Mobile App → Caregiver